06 Validation Methods and Performance Metrics

Markéta Tranberg, mlap@viauc.dk

March 2026

Table of Contents

1 Validation Methods

2 Performance metrics

3 Code example

Why do we need validation?

Data, Model, Performance

- Train learn
- Test
- + Validation

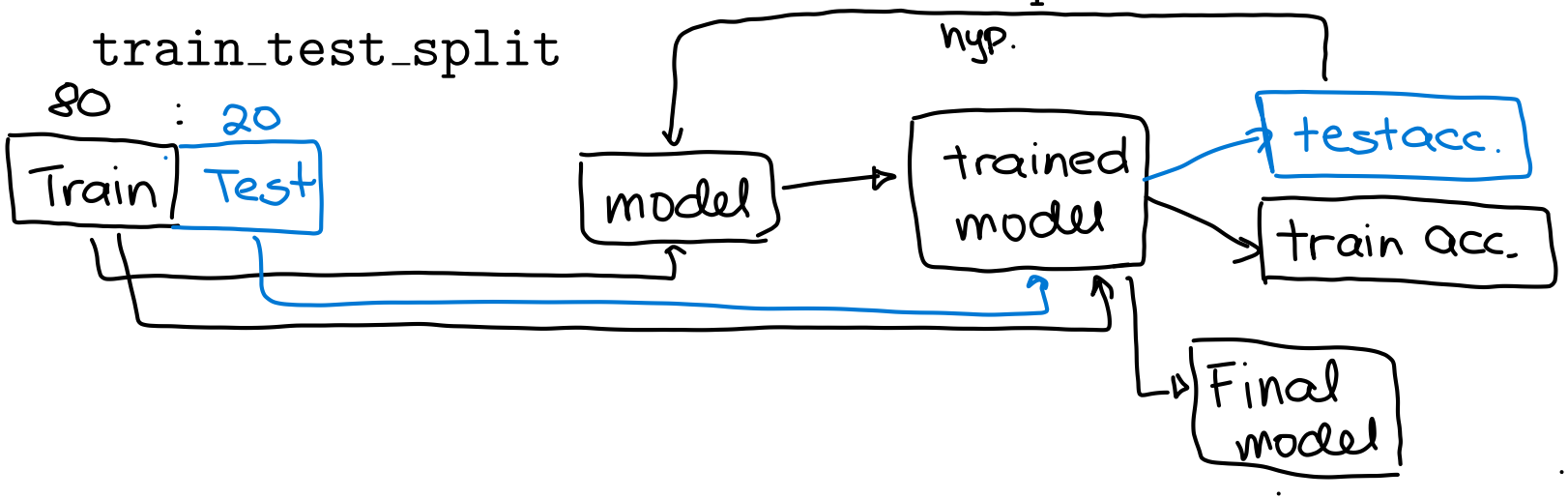
Validation methods

- Train-test
- Train-val-test
- Cross-validation (CV)
- Leave-one-out validation (LOO)

Choice depends on your data (as per usual)

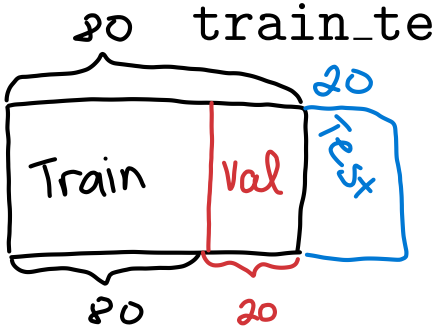
Train-test methodology

- 80-20 split between train and test set
- Compare train and test accuracy
- Tune hyperparameters based on test accuracy
- `from sklearn.model_selection import train_test_split`

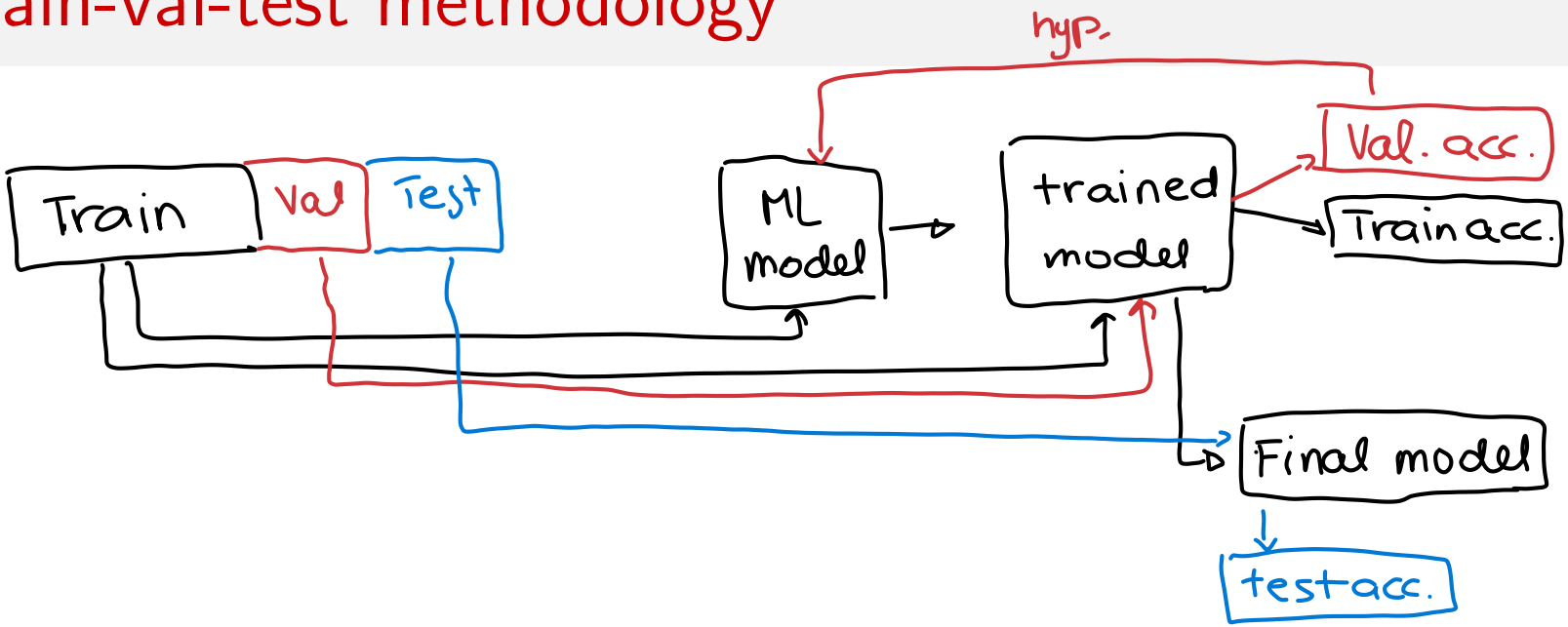


Train-val-test methodology

- First 80-20 train-test split
- Then train split 80-20 into train and validation
- Final 64-16-20 train-val-test split
- If large dataset ($>$ 100k rows), we can split closer to 90-5-5
- Compare train and validation accuracy, tune hyperparameters
- Use test accuracy exactly once on the final model
- `from sklearn.model_selection import train_test_split` used iteratively



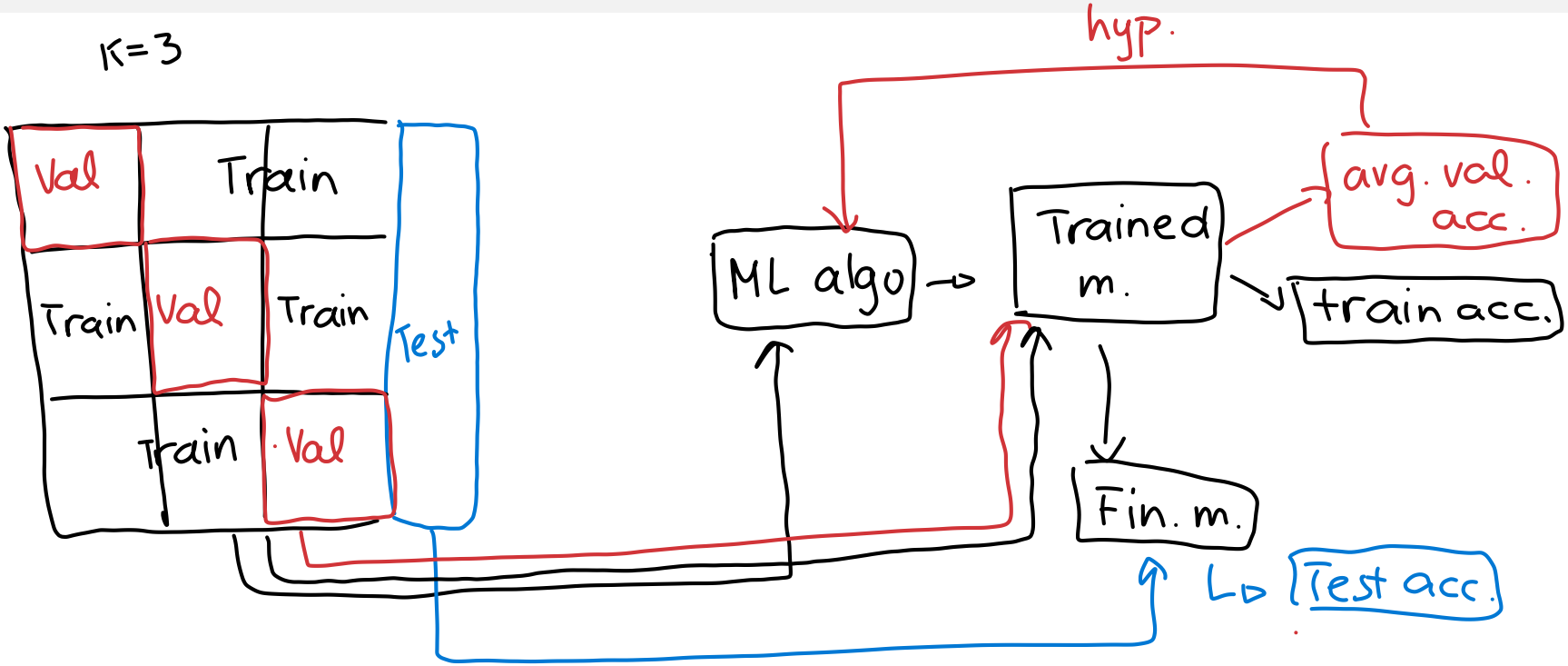
Train-val-test methodology



Cross-validation methodology (CV)

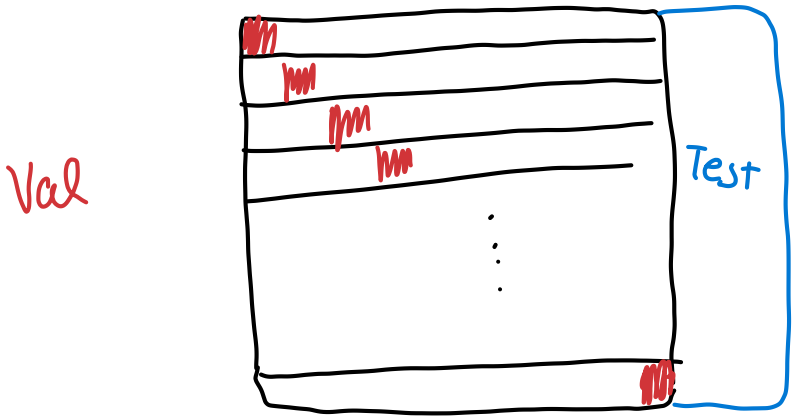
- Split train-test 80-20
- Train test is iteratively split into a series of train-validation
- Compare train and *average* validation accuracy, tune hyperparameters
- Use test accuracy exactly once on the final model
- K-folds: train on $K-1$, validate on 1, repeat K times
- Use case: small datasets, independent rows (not time-series)
- `from sklearn.model_selection import cross_val_score`

Cross-validation methodology (CV)



Leave-one-out cross-validation methodology

- LOO CV
- Same as CV but exactly 1 data point is left out
- $K = n$
- Use case: tiny datasets ($<$ 40 rows)
- Downside: high computational cost, high variance (overlap)
- `from sklearn.model_selection import LeaveOneOut`



Preprocessing and splitting

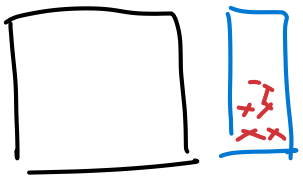
Split test out **before** preprocessing. It should **never** see test sets before it is used.

- Outliers
- Missing values
- Transformation
- Scaling (save scaler)
- One-hot encoding
- ...

Stratification

= maintain the same proportion of *something* (often target) across train-val-test

- reduce sampling bias
- Breast cancer: 1 in 8 women, 1 in 755 men[1]
- `from sklearn.model_selection import StratifiedShuffleSplit`



[1] https://www.nationalbreastcancer.org/breast-cancer-facts/

Table of Contents

1 Validation Methods

2 Performance metrics

3 Code example

Regression performance metrics (recap)

- MSE (mean squared error)
- RMSE (root mean squared error)
- MAE (mean absolute error)
- R^2 (coefficient of determination)

Classification performance metrics

- Confusion matrix
- Accuracy
- Precision and recall (PR curve trade-off)
- F1 score
- ROC curve (=Receiver operating characteristics)

Classification: errors

- Sick and positive test
- Sick and negative test
- Healthy and positive test
- Healthy and negative test

correct

Confusion Matrix

	Predicted Status	
Actual Status	Positive Test	Negative Test
Sick	**True Positive (TP)** (Sick & Positive)	**False Negative (FN)** (Sick & Negative)
Healthy	**False Positive (FP)** (Healthy & Positive)	**True Negative (TN)** (Healthy & Negative)

Confusion Matrix Calculation

No.	Sick	Test res.
1	T	T
2	T	F
3	F	T
4	F	F
5	T	T
6	F	F
7	T	T
8	F	T
9	T	F
10	F	F
11	T	T
12	T	T
13	F	F
14	F	T
15	T	T

	Predicted	
Actual	**Positive**	**Negative**
Sick	TP = 6	FN = 2
Healthy	FP = 3	TN = 4

ifect other,

free day off

Binary classification examples

10:45

Which one do we **minimize**, FN, FP or both?

- Covid test (positive = sick) FN
- Spam filter (positive = spam) FP
- Criminal justice (positive = guilty) FP (both) innocent until proven guilty
- Airport security check (positive = weapon) FN
- Product recommendation (positive = recommendation) both
- Pregnancy test (positive = pregnant) FN

Accuracy

= ratio of correct predictions

$$\frac{TP + TN}{TP + TN + FP + FN}$$

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Precision

$=$ positive predictive value

$$\frac{TP}{TP + FP}$$

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Recall

= sensitivity, true positive rate

$$\frac{TP}{TP + FN}$$

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Sidenote: types of mean

- Arithmetic mean: $\frac{a+b}{2}$ lin. scale
- Geometric mean: $\sqrt{ab}$ multiplicative data (10% increase 50% in stocks)
- Harmonic mean: $\frac{2ab}{a+b}$ rates of ratios driving km/h

F1 score

= harmonic mean of precision and recall

$$\frac{2TP}{2TP + FP + FN} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Predicted

		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Example

Accuracy: 0.9375

Precision: 0.9524

Recall: 0.8696

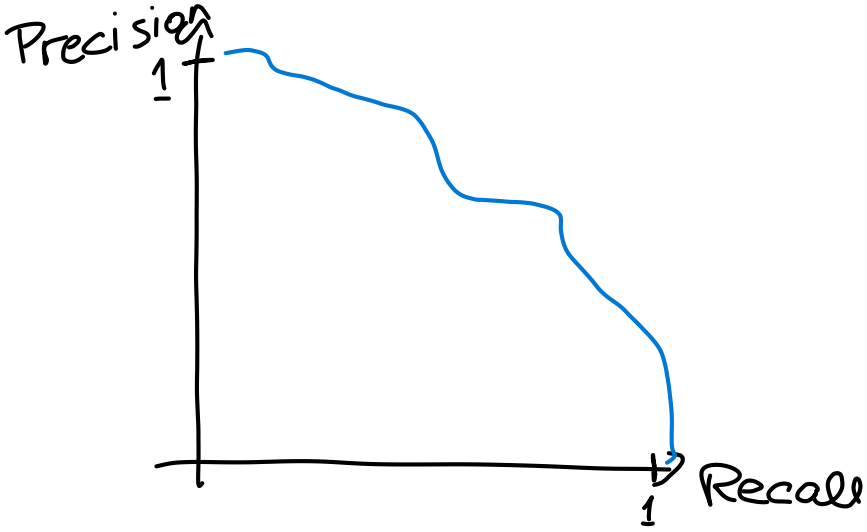
F1 score: 0.9091

		Predicted	
		Pos.	Neg.
Actual	Pos.	100	15
	Neg.	5	200

Precision-recall trade-off

High recall - low precision
Choose the correct one for your use case: set threshold
Low threshold: high recall (catching a virus - minimize FN)
High threshold: high precision ("safe for kids" - minimize FP)

Precision-recall curve: negative correlation



ROC curve

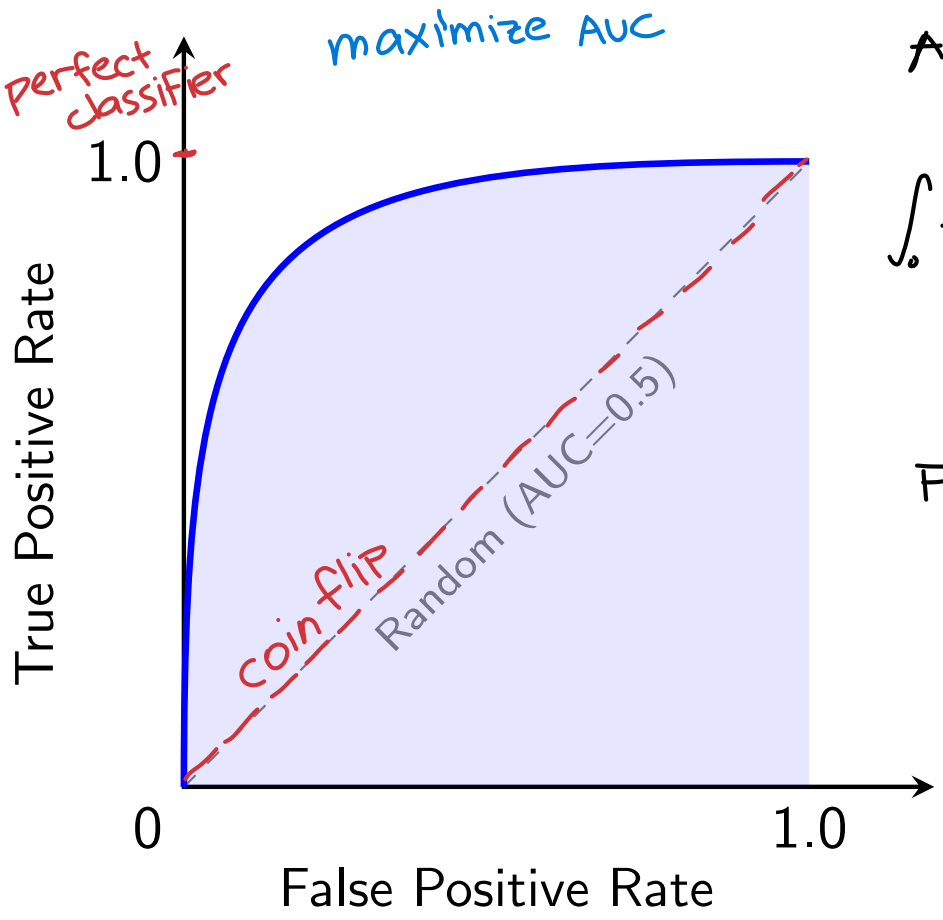
ROC curve plots different thresholds between catching positive and avoiding negatives.

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

		Predicted	
		Pos.	Neg.
Actual	Pos.	TP	FN
	Neg.	FP	TN

ROC curve



AUC = area under curve

$$\int_a^b f(x)\,dx = F(b) - F(a)$$
$$= [F(x)]_a^b$$

F = antiderivative

AUC = area under curve

- $AUC = 1.0$: perfect model
- $AUC = 0.5$: random model (=coin flip)
- $AUC = 0.8 - 0.9$: generally considered good

AUC is scale-independent

PR curve vs. ROC curve

ROC:

- balanced classes
- care equally about positives and negatives
- Ex: rain or no rain (50% chance of each)

PR:

- unbalanced classes: rare event detection
- focus on positive class
- Ex: credit card fraud (99% are legit), spam detection (care about FP than FN)

Often it is a good idea to look at both: ROC for overall capacity, PR for rare positive class (medical screening)

Table of Contents

1 Validation Methods

2 Performance metrics

3 Code example

Code

Github link: `https://github.com/304125/MAL1_S26/tree/master/06_Validation_Methods_and_Performance_Metrics`